Lenguaje de Programación I

Juan Carlos Charfuelan Caipe
correo: juan.charfuelan@uniremington.edu.co

Métodos de Evaluación

- 3 notas Evaluativas - 75%
- 1 nota de seguimiento de actividades - 25%

Fechas Evaluación

- Primera nota - 29/09/2026
- Segunda nota - 13/10/2026
- Nota final - 22/10/2026

Temario

1. Uso de Python
2. Concurrencia y Hilos
3. Sincronización
4. Network y Sockets
5. Acceso Remoto
6. Programación Funcional

Stack Tecnologías



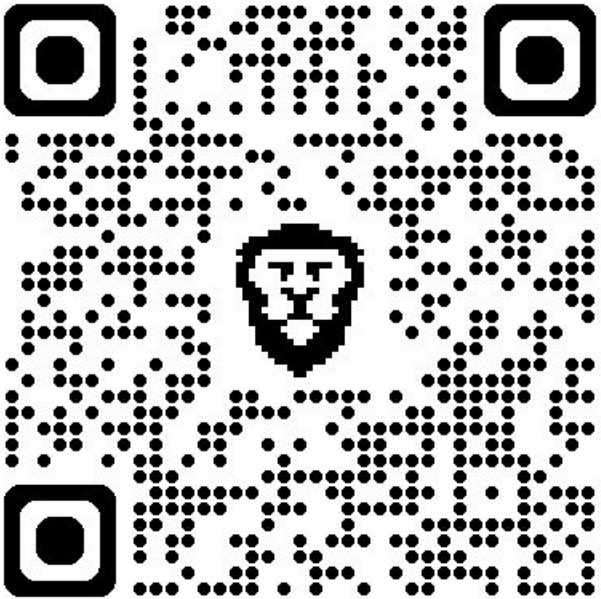








Recursos del Curso



https://github.com/orgs/programming-language-i/repositories

Qué es Python?

Es un lenguaje de programación de alto nivel, diseñado para ser *fácil* de **leer**, **aprender** y **utilizar**.

- **Guido van Rossum:** creo este lenguaje con la idea de construir un lenguaje que permitiera escribir programas de forma clara y sencilla.

Uso de Python

Problemas que se pueden resolver

- Crear aplicaciones web
- Analizar datos
- Desarrollar inteligencia artificial
- Automatizar tareas
- Crear videojuegos
- Programar robots

Python es utilizado en:

- Inteligencia artificial
- Ciencia de datos
- Ciberseguridad
- Desarrollo web
- Automatización
- Internet de las cosas

Git y Github

Que es Git?

Git es un sistema de control de versiones, una herramienta que permite **guardar** y **controlar** los cambios que se realizan en un proyecto, especialmente en el código de un programa.

Git permite llevar un historial de todas las modificaciones que se hacen en un proyecto.

Para qué sirve Git?

Git se utiliza principalmente para:

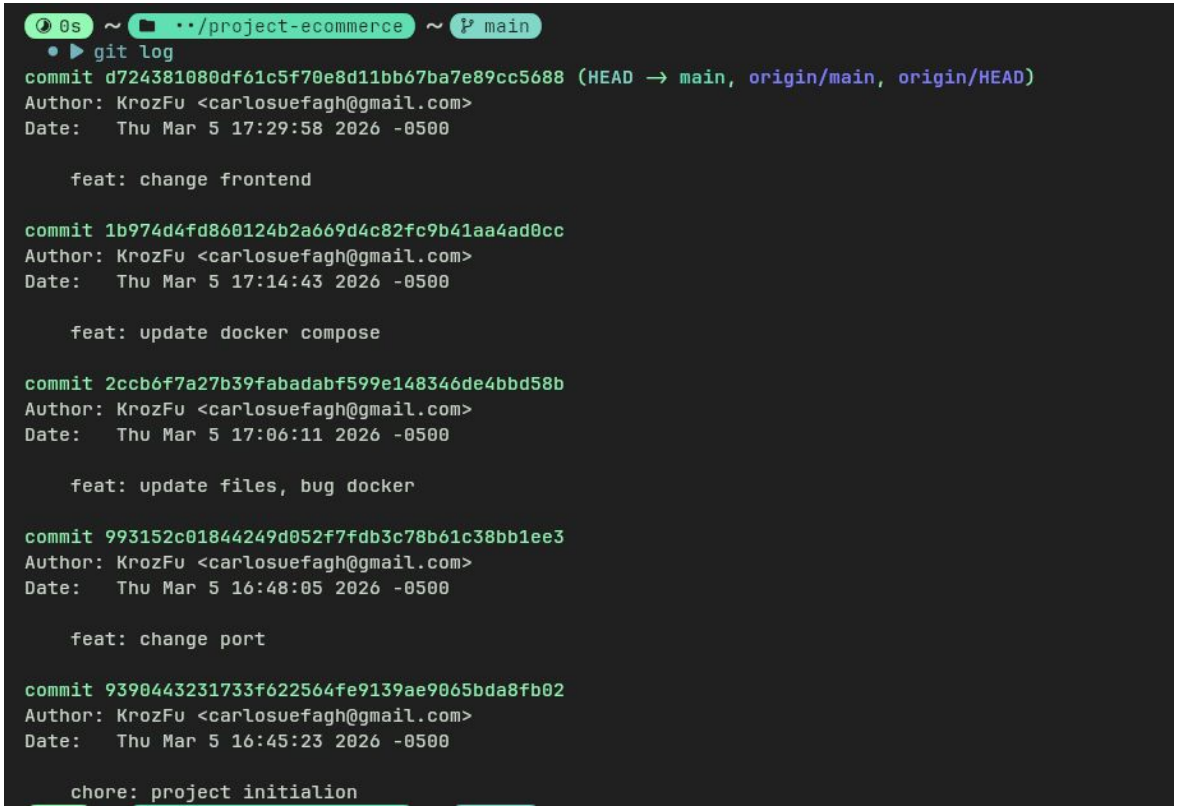
- guardar versiones del código
- trabajar en equipo
- evitar perder cambios importantes
- regresar a versiones anteriores del proyecto
- gestionar el desarrollo de software

Cada cambio que se guarda en Git se llama:

commit

Un commit es básicamente una **fotografía** del proyecto en un momento específico.

Control de Versiones de Git



Que es Github?

Es una plataforma en internet que permite almacenar proyectos que usan Git.

GitHub es un servicio en la nube donde puedes guardar tus repositorios de Git.

- **Repositorio**: Una carpeta que contiene un proyecto y su historial de cambios.

Que nos permite hacer GitHub

- guardar proyectos en línea
- compartir código con otras personas
- colaborar en equipo
- mostrar portafolios de programación
- contribuir a proyectos de código abierto

Diferencias entre Git y Github

Característica	Git	GitHub
Tipo	herramienta	plataforma
Funciona	en tu computador	en internet
Uso principal	control de versiones	almacenamiento y colaboración
Necesita internet	no	sí

Python



Entornos virtuales y paquetes

Es un espacio aislado donde puedes instalar y usar versiones específicas de Python y de librerías (paquetes) sin afectar a otros proyectos o al sistema operativo.

python3 -m venv name-venv

Para qué sirven?

Evitar conflictos: Si un proyecto necesita la versión 3.9 de Python y otro la 3.12, los ambientes virtuales te permiten usar ambas sin problemas.

Organización: Cada proyecto tiene sus propias librerías, sin mezclarse con las de otros proyectos.

Portabilidad: Puedes compartir tu proyecto con otros y ellos instalarán exactamente las mismas librerías.

Manejo ambientes virtuales

- Manejo de dependencias
- Facilitar la instalación de aplicaciones
- Proteger la instalación base de Python
- Tener múltiples versiones de Python
- Seguridad a la hora de manejar dependencias
- Facilita el trabajo colaborativo

Manejo de Virtual Enviroments Windows

main.py +

```
# Crear entorno virtual
python -m venv name_enviroment

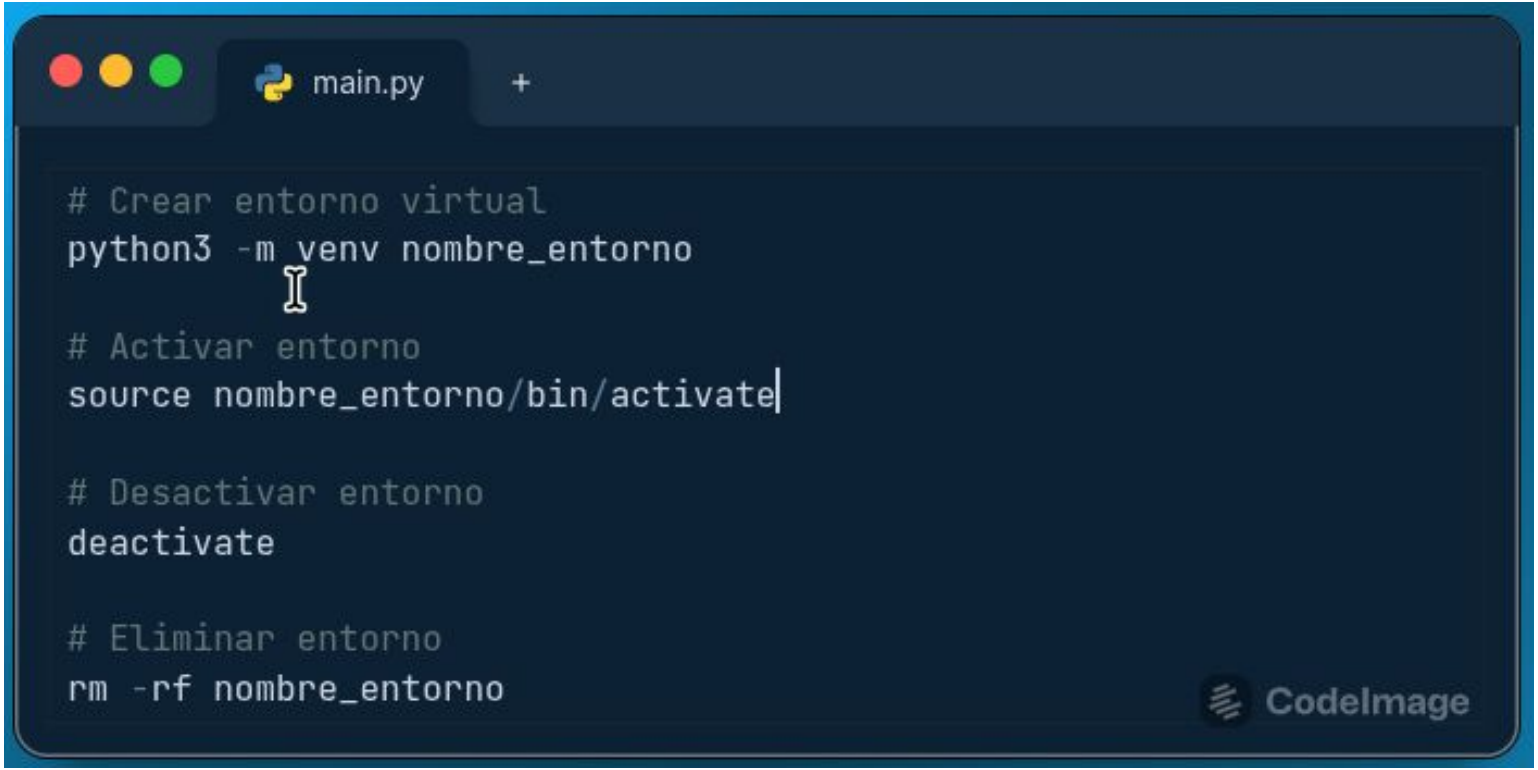
# Activar entorno
name_enviroment\Scripts\activate

# Desactivar entorno
deactivate

# Eliminar entorno (solo borrar la carpeta)
rmdir /s name_enviroment
```

CodeImage

Manejo de Virtual Enviroments Linux



Concepto de Variables

- Contenedores en los que almacenamos datos o valores.
- Permiten manipular, almacenar y modificar informacion en nuestros programas de manera flexible y dinámica.

Nombres de Variables

Los nombre nombres de las variables deben de comenzar con una letra.

No se permiten espacios en los nombres de las variables. En su lugar, puedes usar guiones bajos para separar palabras.

No utilizar palabras reservadas de Python como nombres de variables.

Usar nombres descriptivos y claros para facilitar la lectura y el mantenimiento del código.

Nomenclaturas en Programación

- snake case
- kebab case
- camel case
- pascal case

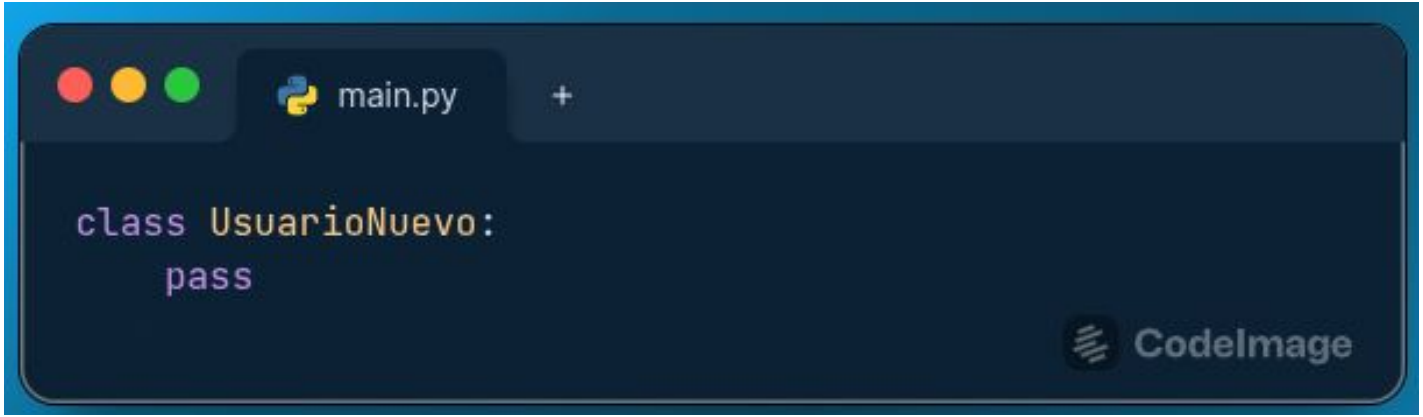
Variables y Funciones

Uso: snake_case (minúsculas con guiones bajos).



Clases

Uso: *PascalCase* (primera letra de cada palabra en mayúscula, sin guiones bajos).



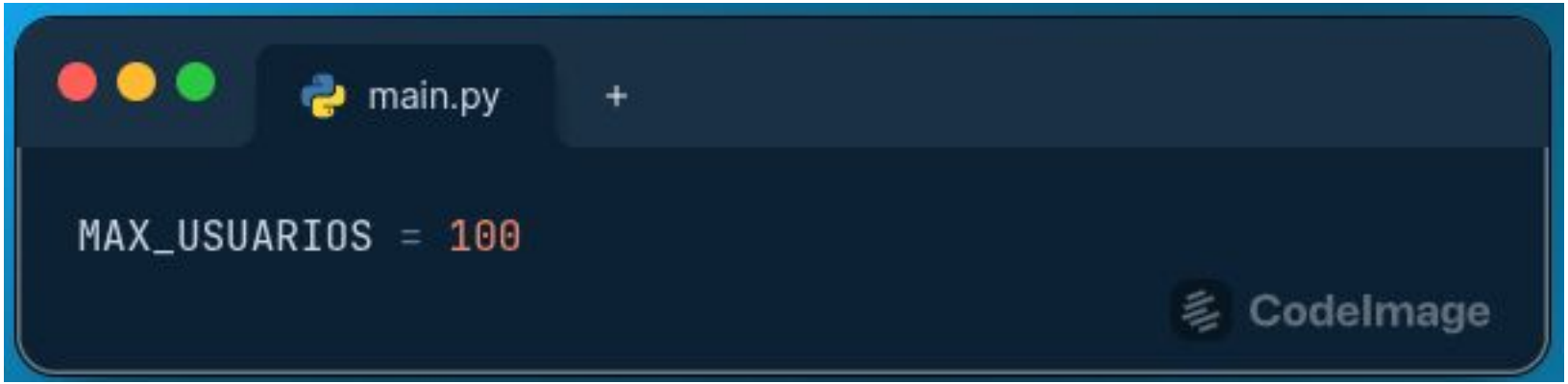
Métodos

Uso: Igual que las funciones, *snake_case*.



Constantes

Uso: *UPPER_SNAKE_CASE* (mayúsculas con guiones bajos).



Nombres privados o protegidos

Uso: Prefijo con un guión bajo _ (para indicar que es "privado" o "protegido").



Tipos de datos básicos en Python (Booleanos)

Verdadero = *True*

Falso = *False*

Tipos de datos básicos en Python (Numeros)

- **Enteros (int):** Son números sin decimales, como 5, 43
- **Flotantes (float)**: Son números con decimales, como 3.14, 0.5, -1.1
- **Complejos (complex):** Son números con una parte real y una parte imaginaria, como 1 + 2j

Operaciones matemáticas básicas

- Suma: a + b
- Resta: a - b
- Multiplicación: a * b
- División: a/b
- División Entera: a//b
- Exponente: a**b
- Módulo: a%b

Conversiones de tipos Numéricos

Casting es el proceso de convertir un valor de un tipo de dato a otro. En Python, esto se hace usando funciones como int(), float(), str(), etc.

Funciones Matemáticas en Python

- **abs(x):** Devuelve el valor absoluto de x.
- **round(x, n):** Redondea x al número más cercano con n decimales (n es opcional).
- **pow(x, y):** Devuelve x elevado a la potencia de y.
- **max(x, y, z, ...):** Devuelve el valor máximo entre los números proporcionados.
- **min(x, y, z, ...):** Devuelve el valor mínimo entre los números proporcionados.
- **sum(iterable):** Suma todos los elementos de un iterable

Concurrencia y Paralelismo

Concurrencia

Concurrencia es la capacidad de un sistema para **gestionar múltiples tareas al mismo tiempo**, alternando entre ellas.

Significa que varias tareas **progresan simultáneamente**, pero no necesariamente se ejecutan exactamente en el mismo instante.

Paralelismo

Paralelismo ocurre cuando varias tareas se ejecutan exactamente al mismo tiempo, utilizando múltiples procesadores o núcleos del CPU.

El paralelismo permite ejecutar varias tareas simultáneamente en diferentes procesadores o núcleos.

Diagramas Ideales

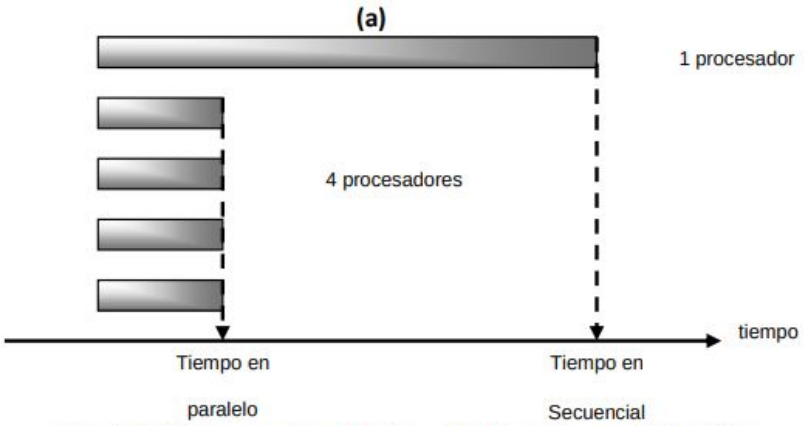


Fig. 1. (a) Esquema ideal del resultado de la paralelización

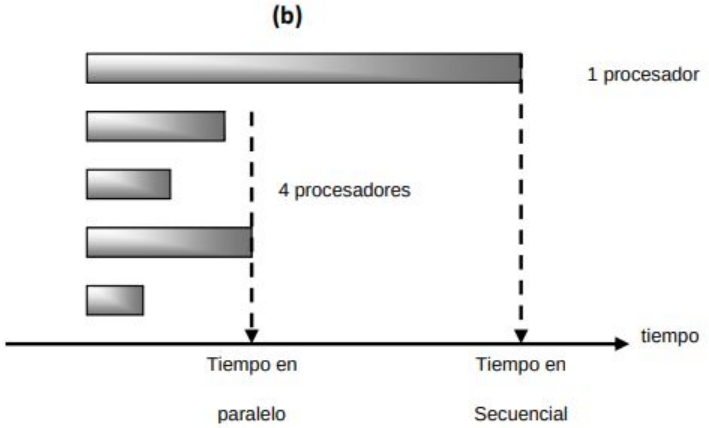


Fig. 1. (b) esquema real, del resultado de la paralelización

https://acortar.link/aretsf

Diferencia

Concurrencia	Paralelismo
Varias tareas avanzan al mismo tiempo	Varias tareas se ejecutan al mismo tiempo
Puede ocurrir con un solo procesador	Requiere múltiples núcleos o procesadores
Las tareas se alternan	Las tareas se ejecutan simultáneamente
Mejora organización del programa	Mejora velocidad de ejecución

¿Qué es un hilo (Thread)?

Un hilo o **thread** es la unidad más pequeña de ejecución dentro de un programa.

Un hilo es un camino de ejecución dentro de un proceso que permite realizar tareas dentro de un programa.

Ciclos de Vida del Hilo

Creado (Newed): Instanciamos el objeto threading.Thread().

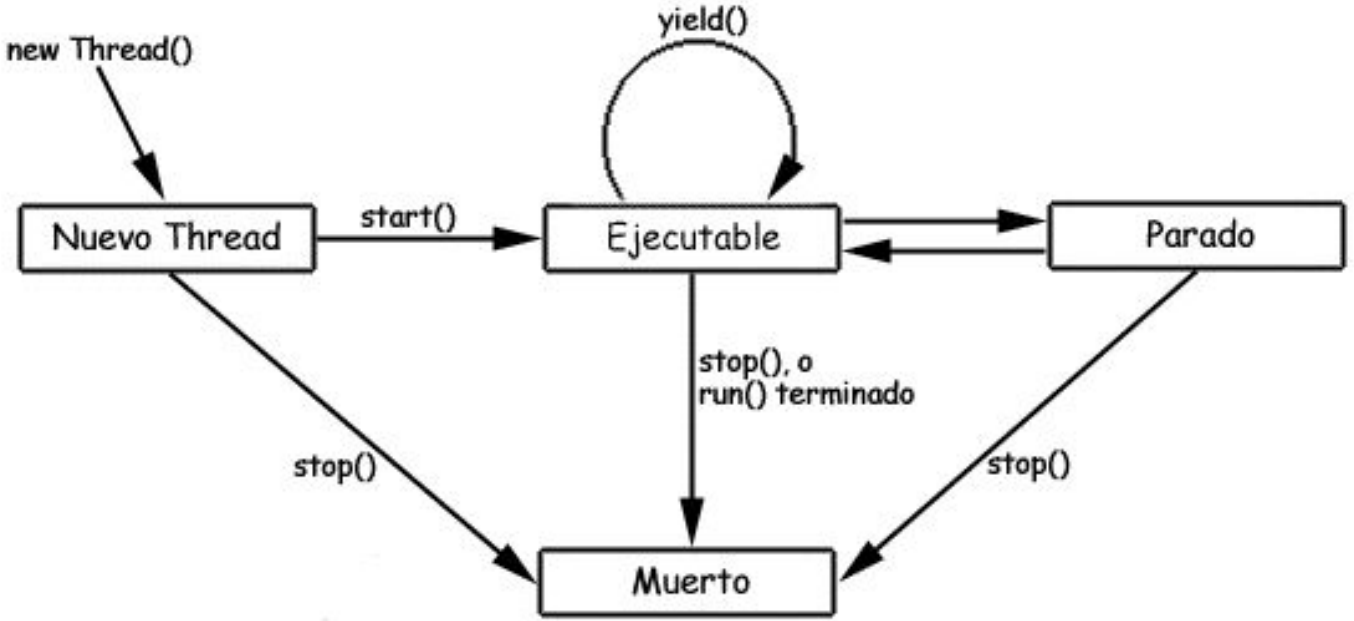
Listo (Ready): Al llamar a .start(), el hilo entra en la cola de ejecución del Sistema Operativo.

En Ejecución (Running): El hilo toma control del procesador.

Bloqueado/Dormido (Blocked): El hilo espera por un recurso o una pausa programada con time.sleep().

Muerto (Dead): El hilo termina su función asignada.

Thread


new Thread()
yield()
Nuevo Thread
start()
Ejecutable
Parado
stop(), o
run() terminado
stop()
stop()
Muerto

Bloqueos o Locks

Condición de Carrera - (Race Condition)

Este problema ocurre cuando **dos o más hilos acceden y modifican simultáneamente un recurso compartido** (como una variable, un archivo o una estructura de datos), y el resultado final depende del **orden en que se ejecutan las operaciones.**

Bloqueos (Locks)

Un **lock** es un mecanismo que garantiza que solo un hilo acceda a un recurso a la vez.

Tipos de bloqueos comunes

1. Mutex
2. Semáforos
3. Monitores
4. Locks reentrantes

Funcionamiento de un Lock

Un lock tiene dos estados:

- libre
- ocupado

Operaciones:

- **acquire()** ➜ toma el lock
- **release()** ➜ libera el lock

Exclusión mutua

La exclusión mutua es un mecanismo que garantiza que solo un hilo o proceso puede acceder a un **recurso compartido al mismo tiempo.**

Mutex

Solo un hilo acceda a una sección crítica a la vez.

Caso real

- saldo bancario
- inventario
- contador de visitas

Semáforos

N hilos accedan al recurso al mismo tiempo.

Caso real

- conexiones a base de datos
- pool de hilos
- control de acceso a recursos limitados

Monitores

Una estructura que combina lock + condición

Caso real

- colas de procesamiento
- sistemas de mensajería
- buffers (streaming, logs)

Locks reentrantes (RLock)

El hilo adquiera el lock varias veces sin bloquearse.

Caso real

- funciones recursivas
- llamadas anidadas
- frameworks complejos

Proceso vs. hilo

	Proceso	Hilo
Memoria	Espacio propio, aislado	Comparte la memoria del proceso
Costo de creación	Alto	Bajo
Comunicación	Mecanismos explícitos: pipes, colas, sockets	Variables compartidas
Fallo	Si cae, los demás procesos siguen	Si el proceso cae, caen todos sus hilos
En Python	`multiprocessing`	`threading`

Elementos Básicos de Network y TCP

Unidad

1. Concurrencia
2. Redes
3. Funcionales

Introducción

Las redes de computadoras permiten la comunicación entre dispositivos para intercambiar información. Hoy en día, casi todas las aplicaciones dependen de redes, desde navegadores web hasta sistemas distribuidos.

Modelo Cliente - Servidor

- **Servidor:** programa que *espera* conexiones en una dirección y un puerto conocido, y responde.
- **Cliente**: programa que *inicia* la conexión hacia el servidor.
- **Asimetría:** el servidor necesita un dirección fija, porque el cliente tiene que saber donde llamar.

Elementos de la arquitectura cliente/servidor

1. Cliente
2. Red
3. Servidor
4. Base de datos



Dirección IP

Una dirección IP es un identificador único asignado a cada dispositivo dentro de una red.

Ejemplos

- 0.0.0.0
- 192.168.1.40
- 128.0.0.0
- 192.0.0.0
- ::1
- 2001:0db8:85a3:0000:0000:8a2e:0370:7334

Direcciones de trabajo del curso

Dirección	Nombre	Qué significa
`127.0.0.1`	*loopback*, `localhost`	Este mismo equipo. El tráfico nunca sale a la red
`0.0.0.0`	"todas las interfaces"	Solo para `bind()` del servidor: aceptar por cualquier interfaz del equipo
`192.168.x.x`, `10.x.x.x`	Privadas	La red local *(LAN)*; no se enrutan en Internet

Conexión

Puerto:

Un puerto es un canal lógico de comunicación dentro de un dispositivo.

Socket:

Un socket es un punto de conexión entre dos dispositivos en una red.

Funciones

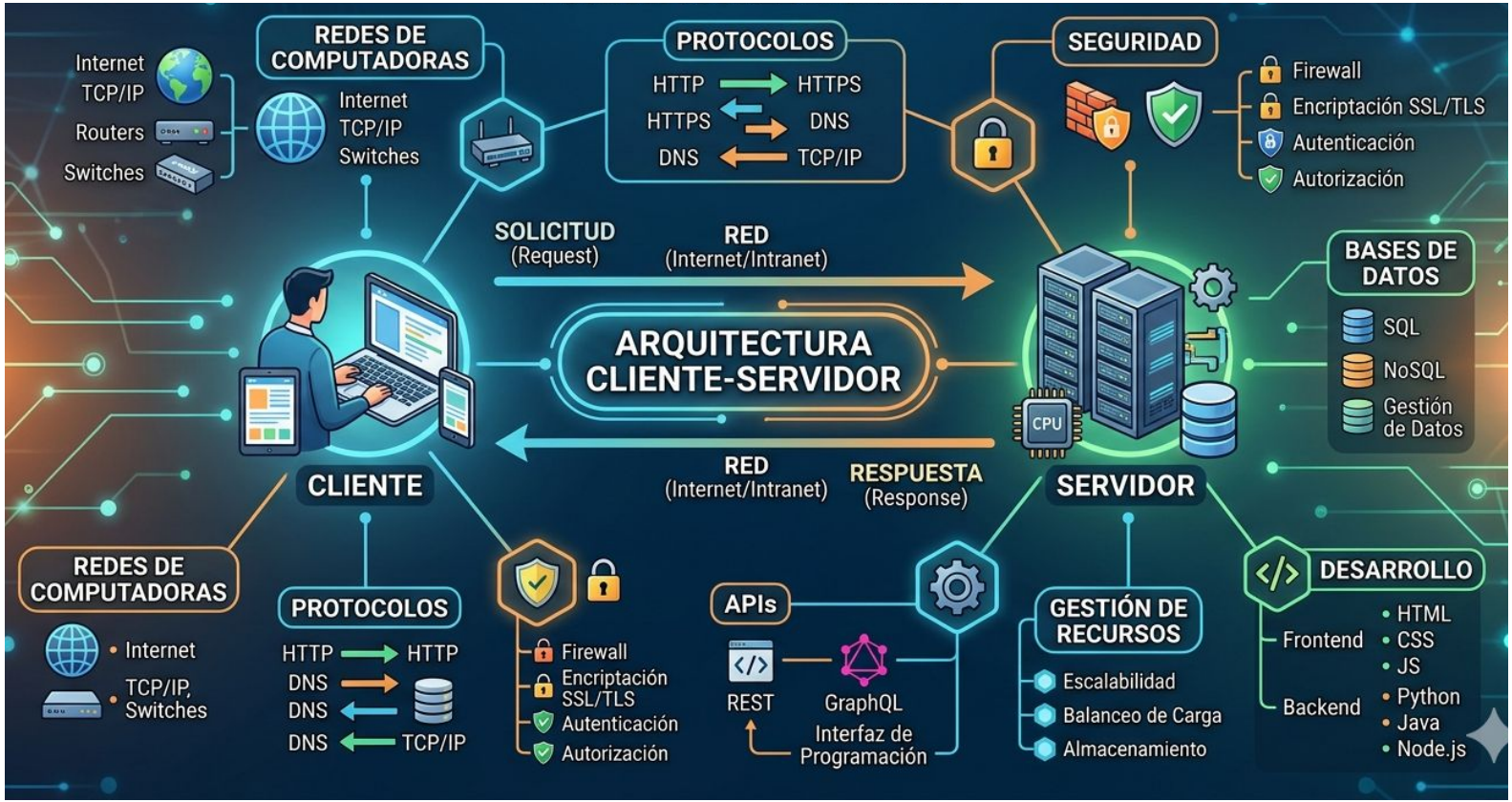
Cliente:

- Administrar la interfaz de usuario
- Interactuar con el usuario
- Procesar las aplicaciones y validar local
- Recibir los resultados o respuestas del servidor
- Formatear resultados
- Generar los requerimientos de las bases de da

Servidor

- Aceptar los requerimientos de los clientes
- Procesar los requerimientos de las bases de datos
- Formatear los datos
- Transmitir a los clientes los datos
- Procesar la lógica y hacer las validaciones

Arquitectura



Socket

Un **socket** es una abstracción o **puerta de comunicación**, que permite que procesos se comuniquen a través de la red.

TCP

Se asegura de que cada bit llegue a su destino, en el orden correcto y sin errores.

- *Orientado a conexión:* Antes de enviar datos, cliente y servidor deben saludarse y confirmar la conexión (proceso llamado Three-way Handshake).
- *Fiabilidad:* Si un paquete se pierde por el camino, TCP pide que se reenvíe.
- *Orden:* Los datos llegan exactamente en el orden en que se enviaron.
- *Uso común:* Navegación web (HTTP/HTTPS), correos electrónicos (SMTP) y transferencia de archivos (FTP).

UDP

El **UDP** es el protocolo "*veloz*". No pierde tiempo preguntando si el otro está listo; simplemente lanza la información.

- *Sin conexión:* No hay saludo previo. El emisor envía paquetes (datagramas) y espera que lleguen.
- *Sin garantías:* Si un paquete se pierde, se perdió para siempre. No hay reenvíos.
- *Velocidad:* Al no tener que verificar errores ni mantener el orden, es muchísimo más rápido y ligero.
- *Uso común:* Streaming de video (si se pierde un frame, no importa tanto como que se trabe el video), juegos online y llamadas de voz (VoIP).

AF_INET

AF_INET es la familia de direcciones IPv4.

- Define el tipo de dirección que se va a usar: aquí, direcciones IP v4

AF_INET6 es la familia de direcciones IPv4

- Define el tipo de dirección para IPv6.

SOCK_STREAM

SOCK_STREAM define el tipo de socket basado en flujo confiable, es decir, TCP.

- *Es orientado a conexión*: primero connect (cliente) y listen/accept (servidor).
- Proporciona flujo de bytes ordenado, fiable y sin duplicados, con control de errores, reenvíos y flujo (TCP).

SOL_SOCKET

SOL_SOCKET es una constante de nivel que indica que la opción de socket se aplica al propio socket, no a un protocolo específico como TCP o IP.

- Sirve como “capa base” para configurar opciones generales.

SO_REUSEADDR

SO_REUSEADDR es una opción de socket que permite reusar rápidamente una dirección **(IP + puerto)** que antes estaba ocupada por otro socket que aún está en estado **TIME_WAIT**.

- Es muy útil en servidores TCP, porque evita el error “Address already in use” cuando se cierra el servidor y se vuelve a lanzar poco después.
- Básicamente, permite que el nuevo servidor vuelva a usar la misma IP y puerto aunque el socket anterior aún está “limpiando” la conexión.

recvfrom

recvfrom devuelve una tupla con dos elementos:

R= (datos, direccion)

datos: Un objeto de tipo bytes con el contenido del mensaje.

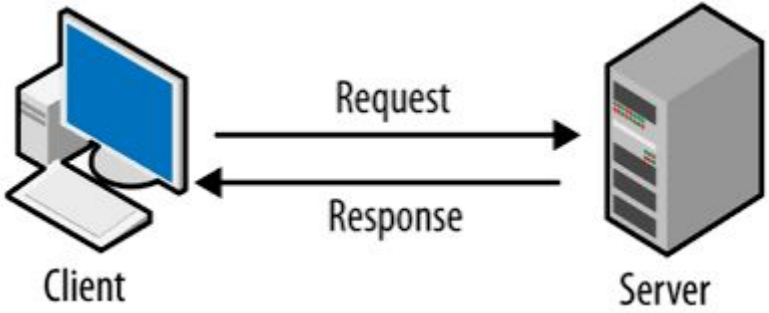
dirección: Una tupla (IP, Puerto) del emisor.

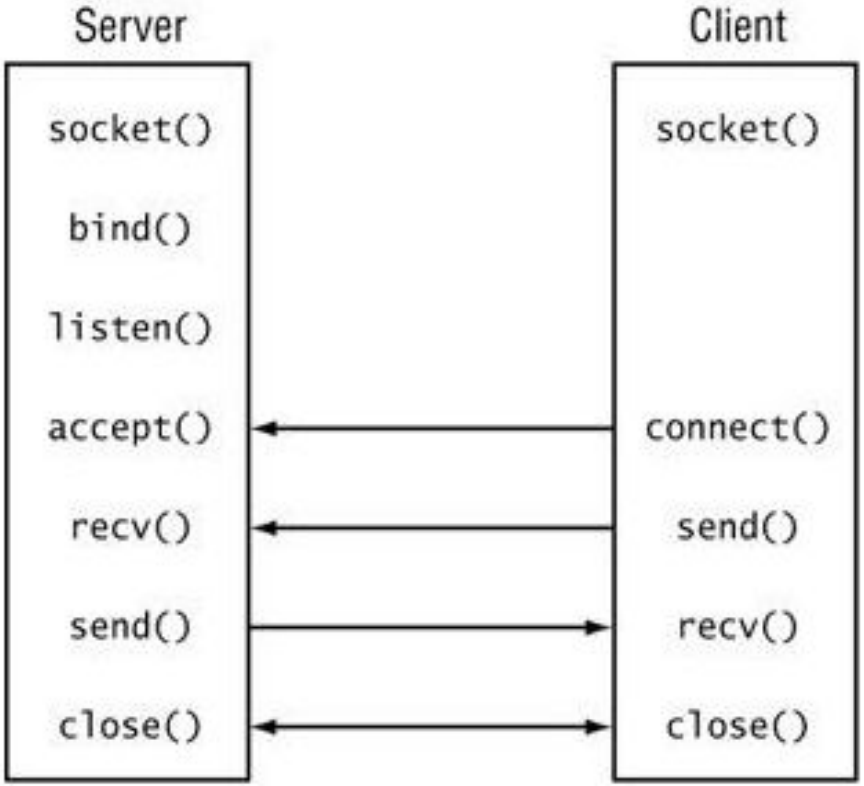
Recibir datos junto con la dirección del dispositivo que los envió.

Anatomía del socket

Servidor	Cliente	Qué hace
`socket()`	`socket()`	Crea el extremo *(familia + tipo)*
`bind((ip, puerto))`	—	Fija la dirección del servidor
`listen()`	—	Lo marca como pasivo: acepta conexiones
`accept()`	`connect((ip, puerto))`	**Bloquea** hasta que llega un cliente · inicia la conexión
`recv()` / `send()`	`send()` / `recv()`	Intercambian bytes
`close()`	`close()`	Liberan la conexión

Cliente - Servidor

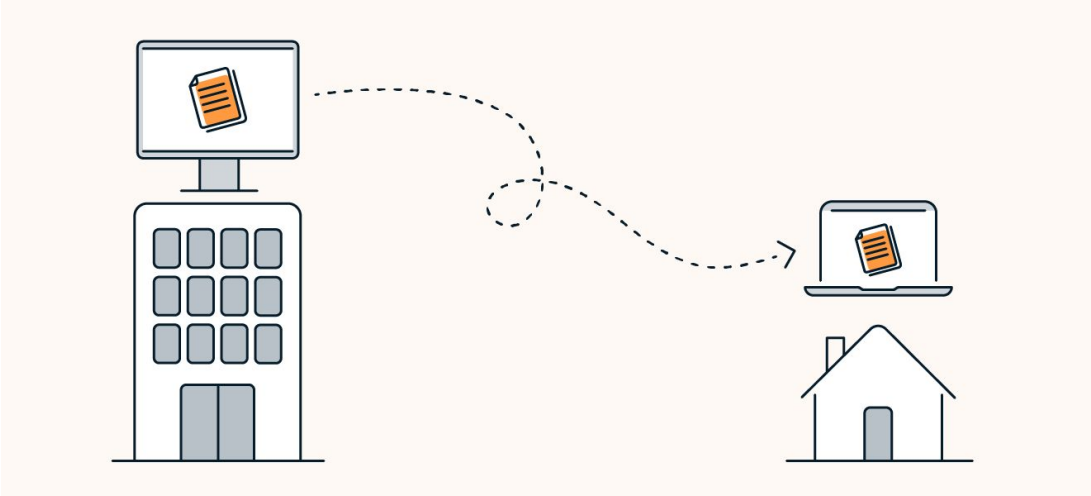




Acceso Remoto

Acceso Remoto

Un **objeto** remoto es un objeto que no está en tu programa, sino en otra máquina o servidor, pero lo usas como si fuera local.



Serialización

Nos va a permitir tomar el estado de un objeto y colocarlo en un medio de almacenamiento.

- Uso de **Pickle** en python

Proceso

- **Objeto** ⇒ bytes (*serialización*)
- Envío por red
- **bytes** ⇒ objeto (*deserialización*)

Uso de Pickle

pickle.dump()

- Esta función recibe dos parámetros: el objeto con los datos a serializar, y el archivo donde se van a escribir.
- *pickle.dump(obj, f)*

pickle.load()

- Esta función recibe como parámetro el archivo de donde se van a leer los datos, y devuelve el objeto deserializado.
- *datos = pickle.load(f)*

Seguridad de Pickle

Pickle solo entre *procesos que confían mutuamente*. Para datos que vienen de fuera, **JSON**.

JSON: interoperable, legible, seguro

- Texto plano, legible, soportado por cualquier lenguaje. No ejecuta nada al leerlo: **json.loads** solo produce **dict**, **list**, **str**, **números**, **bool** y **None**.
- **ensure_ascii=False**: sin él, ñ sale como \u00f1. Con él, el texto queda legible y se guarda en **UTF-8**

JSON: interoperable, legible, seguro

Limitaciones, que se pagan:

Python	Qué pasa en JSON
`tuple`	Vuelve como `list`
`set`, `bytes`, `datetime`	`TypeError`: no son tipos de JSON
Claves de `dict` que no son `str`	Se convierten a `str`
Una instancia de tu propia clase	`TypeError`: hay que convertirla a `dict`